

Document number: P2285R1
Date: 2026-02-23
Audience: EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Tomasz Kamiński <tomaszkam at gmail dot com>

Are default function arguments in the immediate context?

This paper deals with

- default function arguments and default member initializers in the immediate context and
- default member initializers affecting whether a defaulted default constructor is deleted.

It is motivated by two observations:

1. Compilers disagree and the standard has no clear stance (at least in the former case) on whether the evaluation of default function parameters and default member initializers is in the immediate context of the enclosing construct.
2. Form the users' perspective (both these who call functions/create objects, and who define them) it is useful to have default function arguments and default member initializers in the immediate context. In fact, many users already assume that it is the case, and some compilers reinforce this assumption.

We explore the current situation, the motivation for the change, and the implementation challenges.

This paper addresses [\[CWG2296\]](#), and partially addresses [\[US54-100\]](#).

0. Revision history

0.1. R0 → R1

1. Changed the tone from a firm proposal to analysis.
2. Added discussion of implementation challenges.
3. Added the analysis of lambdas in default function arguments.
4. Added to the scope the case of default member initializers affecting whether a defaulted default constructor is deleted.

1. Motivation

The motivation for default function arguments is to enable two different function invocation forms while providing a single function declaration:

```
template <class T>
struct C
{
    explicit C(T = 1);
};

C<int> c;    // works
C<int> c(1); // works
```

The purpose of concepts and type traits is to determine if certain constructs are valid, for instance to avoid using ill-formed constructs:

```
std::cout << std::constructible_from<C<std::string>>;
```

Today, compilers do not agree on the outcome:

GCC	Clang	MSVC	ICX
15.2	21.1	14.44	2025.3
outputs 0	ill-formed	outputs 1	ill-formed

This divergence makes it difficult to write portable code.

One could reasonably argue that a person writing the template should know the range of allowed types, and should not provide default initializers that don't work. It is a bad library design. But consider this specification copied almost 1-to-1 from the Standard Library:

```
struct Map
{
    explicit Map(Range&&, Hash, Equal, Allocator);

    explicit Map(Range&& c, Hash h, Equal e)
        requires default_initializable<Allocator>
        : Map(c, h, e, Allocator()) {}

    explicit Map(Range&& c, Hash h)
        requires default_initializable<Equal>
            && default_initializable<Allocator>
        : Map(c, h, Equal(), Allocator()) {}

    explicit Map(Range&& c)
        requires default_initializable<Hash>
            && default_initializable<Equal>
            && default_initializable<Allocator>
        : Map(c, Hash(), Equal(), Allocator()) {}
};
```

If function arguments were considered in the immediate context of the function/constructor invocation, the same effect could be achieved with a single constructor:

```
struct Map
{
    explicit Map(Range&&, Hash = Hash(), Equal = Equal(), Allocator = Allocator());
};
```

There is a related issue: how the validity of default member initializers is checked when the default constructor is explicitly defaulted.

```
template <typename T>
struct A
{
    A() = default;

    T mem = T{};
};
```

It is so natural for programmers without the arcane knowledge of template specialization instantiation, or even for programmers with that knowledge when they are caught off guard, to assume that this code means "if the expression `T{}` is ill formed, make the default constructor deleted". This very case has been a subject of a number of LWG issues, which illustrates that even LWG experts fall into this trap.

In this case, the Standard is pretty clear: only the presence/absence, not the validity of the initializer affects whether the default constructor is deleted. However, not all compilers comply.

The current compiler status for test cases:

```
template <typename T>
concept DefInitParen = requires { T(); };

template <typename T>
concept DefInitBrace = requires { T{}; };

std::cout << std::is_default_constructible_v<A<NoDefault>>;
std::cout << std::default_initializable_v<A<NoDefault>>;
std::cout << std::constructible_from_v<A<NoDefault>>;
std::cout << DefInitParen<A<NoDefault>>;
std::cout << DefInitBrace<A<NoDefault>>;
```

Test case	GCC 15.2	Clang 21.1	MSVC 14.44	ICX 2025.3	Present mandate
is_default_constructible_v	outputs 0	ill-formed	outputs 1	ill-formed	output 1
default_initializable	outputs 0	ill-formed	outputs 0	ill-formed	output 1
constructible_from	outputs 0	ill-formed	outputs 1	ill-formed	output 1
DefInitParen	outputs 0	ill-formed	outputs 1	ill-formed	output 1
DefInitBrace	outputs 0	ill-formed	outputs 1	ill-formed	output 1

2. Properties of default function arguments

This section lists the properties of functions with default function arguments, which shapes how users think about them and what expectations they develop.

2.1. Function call

```
int f(int i, int j = make_j());
```

The above declaration enables two forms of function calls:

```
f(0);
f(0, 1);
```

This feels almost like we were dealing with two functions. The point is, the user could say, "I can call `f` with one argument and I can call it with two arguments".

2.2. Function address

The following case works:

```
int f(int i, int j = make_j());
auto p = f;
```

Meaning that while you can make function calls as if they were two functions, it is still a single function.

This fails to compile:

```
using unary_fun = int (*)(int);
unary_fun p = f;
```

So the default function arguments are added in the function call but not in pointer casting. This also illustrates that reasoning "if I add a new parameter to my function and give it a default function argument, my program will not break" is false.

2.3. Function's source location

Given the following code:

```
int f(int i, int j = std::source_location::current().line()); // Line A

int main()
{
    f(1); // Line B
}
```

The value of parameter `j` is initialized to line B, where the default function parameter is *used* rather than where it is declared.

2.4. How names are looked up

The names used to define the default function argument are looked up from the point where the function is declared, rather than where the argument is used.

Consider the following example:

file main.cpp	file lib.hpp
<pre>#include "lib.hpp" int main() { for (int width : {1, 2, 3}) foo(width); }</pre>	<pre>const int width = 4; void foo(int val, int w = width) { std::cout << val << w; }</pre>

The output is 142434 (not 112233).

2.5. Lambdas in default arguments

Consider the following contrived case suggested by Richard Smith ([63391]).

```
int counter = 0;
template<typename T> auto id = ++counter;
template<typename T> int f(int n = id<decltype([]){})> { return n; }
```

It has the ability to count the unique lambda types generated from the unevaluated expression `[]{}.` Given the following sequence of calls to `f`, how many unique lambda types are created?

```
f<int>();
f<int>();
```

All compilers — Clang, GCC, MSVC, ICX — agree that only one unique type is generated, irrespective of the number of calls. This is a requirement of the Itanium ABI. The C++ Standard leaves this implicitly unspecified.

3. Usability Analysis

3.1. Basic SFINAE

Originally (C++98), SFINAE was introduced to prevent the case where during overload resolution a "remote", unrelated function template declaration spoils the usage of the obvious best candidate. Consider this example by David Vandevoorde:

```
template <typename T>
T f(T, typename T::Ptr = 0); // unrelated inclusion

int f(int);                // the obvious local candidate

int r = f(42);             // works due to SFINAE
```

The same "unrelated overload" problem can manifest when a default function arguments are at play:

```
template <typename T>
T f(T v, T u = T::def()); // unrelated inclusion

int f(long);              // the obvious local candidate

int r = f(1);             // should work
```

Current compiler status:

Test case	GCC	Clang	MSVC	ICX
	15.2	21.1	14.44	2025.3
f(1)	ill-formed	ill-formed	ill-formed	ill-formed
f(1L)	ok	ok	ok	ok

3.2. Consistency with default member initializers

Ever since C++ got the *parenthesized aggregate-initialization*, this in combination with default member initializers often looks indistinguishable from invoking a constructor with default parameters. Consider:

```
template <typename T>
struct S
{
    T x;
    T y = T{};

    // explicit S(T x, T y = T{}) : x{x} y{y} {}
};

S x (1);    // ok
S y (1, 2); // ok
```

The user code behaves the same if we provide the commented-out constructor of S. Because of this similarity, for the sake of having a simple conceptual model, it is desirable to have the usage of default member initializers also in the immediate context of the object initialization.

Currently, we also have a divergence in behavior between compilers. Consider the following test:

```
struct NoDefault
{
    explicit NoDefault(int) {}
};

int main()
{
    std::cout << std::constructible_from<S<NoDefault>, NoDefault>;
}
```

Compilers' outcome:

GCC	Clang	MSVC	ICX
15.2	21.1	14.44	2025.3
outputs 0	ill-formed	outputs 0	ill-formed

4. Implementability

From the presented compiler tests, it looks like Clang and ICX (using EDG frontend) have taken the same approach: because both default function arguments and default member initializers are treated as *separately instantiated entities*, as other such entities, they are treated as not being in the immediate context: having to instantiate a template, implies a hard error upon template argument substitution failure. They do instantiate the template, and therefore any check via concepts or type traits ends in a hard compilation error.

GCC indeed tries to determine the validity of the default function arguments in order to give the answer. The template instantiation that needs to happen here is either not performed, or its effects still not trigger the hard error. In the case of default member initializer's invalidity causing the default constructor to be deleted, GCC goes as far as to violate the letter of the Standard ([\[class.default.ctor\]](#)) in order to give the programmer a useful result.

MSVC gives different answers depending on which tool for validity testing is used. In the case of default function arguments, the mechanism end up being, "if the default function argument exists assume it will be valid". No template instantiation is performed.

Both Clang and EDG implementers tie the notion of *separately instantiated entities* to being not in the immediate context. If the language started mandating that default function arguments and default member initializers were in the immediate context, this tie would be broken, and the intuition the compiler developers developed, as well as a lot of educational materials on templates for the programmers, would get invalidated.

Clang developers raised a concern about the interaction between lambdas in default function arguments and situations where the template argument substitution fails in one location but succeeds in another. Consider this example from Richard Smith:

```
template<typename T> int *g(int *p = ([]{}, T(), []{ static int n; return &n; }())) { return p; }

template<typename T> void test(decltype(g<T>()));
template<typename T> void test(...);

int *h() {
    struct A;
    test<A>(0);    // substitution #1
    struct A {};
    return g<A>(); // substitution #2
}
```

The first substitution fails, but it leaves side effects in the form of an instantiated function-static variable. This may have an impact on the second substitution (one that succeeds). In order to avoid this impact, one known implementation strategy is that each use of the default argument would have to trigger a unique lambda type, meaning that each use is associated with different name mangling. This might have an impact on the ABI. Clang doesn't need to address this problem today, because it treats the default function arguments as non-immediate context, and simply errors out upon the substitution failure.

The feedback from compiler vendors during and after the Kona 2025 meeting was as follows. GCC developers didn't report concerns. Clang developers wanted more time to verify if there is ABI impact. EDG representation was of the opinion that default function arguments in the immediate context are implementable, but not desirable, while default member initializers might not be implementable at all. They requested more time for investigation. An MSVC representative said that default function arguments should not be harder to implement than the `source_location` magic; default member initializers, while harder, should also be doable.

4. Our Recommendation

Our first recommendation, provided that the implementers deem it feasible, is to have the Standard match the expectations of the programmers. That is, the evaluation of default function arguments shall:

- be in the immediate context of the enclosing construct,
- remain a separately instantiated entity for other purposes,
- not create a separate lambda type for each place where a default function argument containing a lambda is used.

The same recommendation applies to the default member initializers in the immediate context of the expressions involving the class.

Similarly, we recommend that a defaulted default constructor be declared as deleted when the involved default member initializer is ill-formed.

Otherwise, if the above implementation is deemed unimplementable or uneconomical, our recommendation is to at least mandate a predictable behavior:

- still, not create a separate lambda type for each place where a default function argument containing a lambda is used,
- have the default function arguments and default member initializers be outside the immediate context of the enclosing function call,
- retain the current behavior in [\[class.default.ctor\]](#).

Rationale: without default function arguments being considered the immediate context:

1. The use case with the constructor overload set in STL containers shows the default function arguments to be a failed attempt. Often, we want a language feature to be consumed by the Standard Library as a proof that a feature is usable. If the Standard Library cannot consume the feature, this is a forecast that the users may also be unable to consume it.
2. Without it, the primary goal of SFINAE is not achieved. (See section [3.1](#).)

5. Wording

The wording will be provided as a separate effort aiming at defining "the *immediate context* of a construct".

6. Acknowledgments

Tim Song indicated the full scale of the problem with default function arguments, which motivated the scope and shape of this paper.

We are grateful to Hubert Tong, David Vandevoorde, Richard Smith and Ville Voutilainen for their valuable input.

7. References

- [CWG2296] — Jens Maurer, "C++ Standard Core Language Active Issues, Revision 111", issue #2296, (https://www.open-std.org/jtc1/sc22/wg21/docs/cwg_active.html#2296).
- [N4892] — Thomas Köppe, "Working Draft, Standard for Programming Language C++" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf>).
- [P0348R0] — Andrzej Krzemiński, "Validity testing issues" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0348r0.html>).
- [P1073R2] — Richard Smith, Andrew Sutton, Daveed Vandevoorde, "Immediate functions" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1073r2.html>).
- [63391] — llvm/llvm-project GitHub issue, "clang++: Surprising SFINAE behavior for default function arguments" (<https://github.com/llvm/llvm-project/issues/63391>).